MAD-2 Project Report

Student Details

Name: Jatin Yadav , Roll Number: 22f2001018 , Email: 22f2001018@ds.study.iitm.ac.in , Current Status in BS degree: Diploma level student

Project Details

Question Statement

- **Overview :** A multi-user platform connecting customers with service professionals, managed by an admin. The application enables:
  - **Admins** to oversee users, approve professionals, and manage service offerings.
  - **Customers** to search, book, modify, and review services by location or type.
  - **Service Professionals** to accept/reject requests and update service statuses.
- **Tech Stack**
  - **Backend:** Flask (API), SQLite (DB), Redis (caching), Celery (batch jobs)
  - **Frontend:** Vue.js, Bootstrap
  - **Auth:** Flask-Security or JWT

Approach

- Backend: Flask & Celery
  - API & Routes: Defined in routes.py to handle user (customer/professional) and admin operations.
  - Database: Managed with Flask-SQLAlchemy, defining models for users, services, and service requests, with appropriate relationships.
  - Task Queue: Celery is used for background jobs, with scheduled tasks configured in celery_schedule.py and task execution handled in tasks.py.
  - Mail Service: Implemented in mail_service.py for automated notifications.
- Frontend: Vue.js & Bootstrap
  - Component-Based Structure: Vue.js components for different roles (admin, customer, professional) are organized under pages/.
  - Navigation: Managed through Navbar.js.
  - Service Requests: Customers can create, update, and track requests (customer_requested_service.js).
  - Admin Dashboard: Features management pages (admin_dashboard.js, admin_service.js).
- Additional Features
  - Authentication: Flask-Security for user authentication and role-based access control.
  - Caching & Performance: Redis used for caching frequently accessed data.
  - Task Scheduling: Celery with periodic tasks for automated notifications and reporting.
  - Static Assets: Stored under static/ for styling and front-end resources.

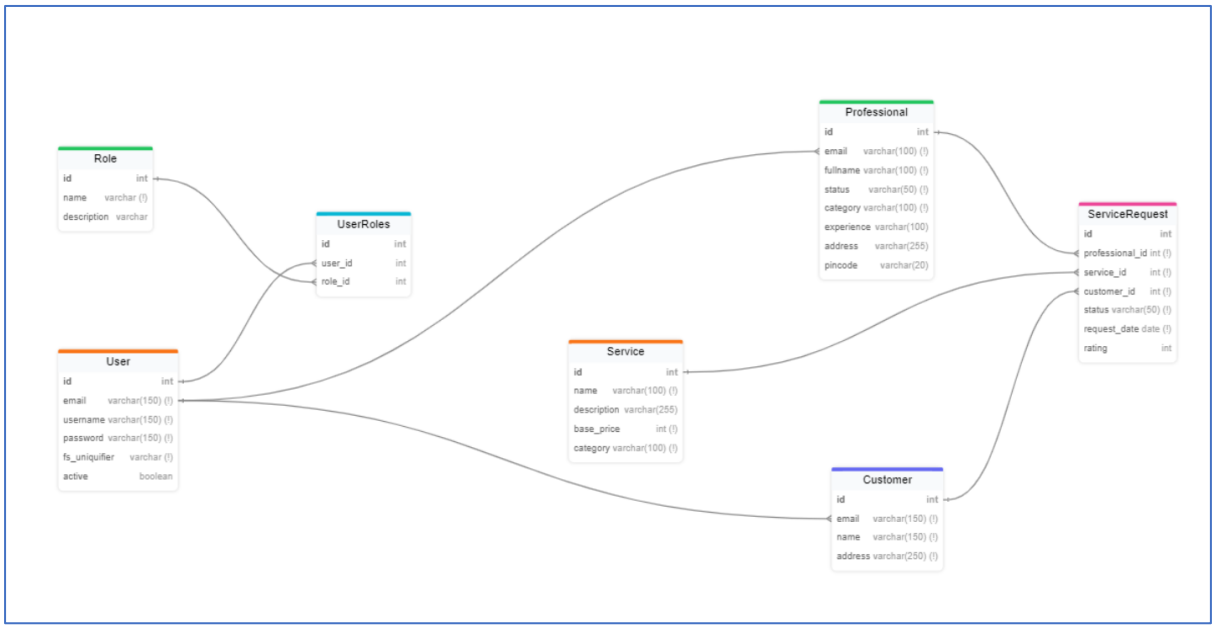
Implementation of program

- First Download the required python libraries by “pip install -r requirements.txt”
- Then move to current directory in and run command “python app.py” , “celery -A app:celery_app beat -l INFO” , “celery -A app:celery_app worker -l INFO” , “~/go/bin/MailHog” and “sudo service redis-server start”
- And open url: http://127.0.0.1:5000/ , http://localhost:8025/# in 2 separate chrome tabs.

Frameworks and Libraries Used

- Backend (Flask & Security)
  - Flask (Core framework), Flask-RESTful (API development), Flask-SQLAlchemy (ORM), Flask-WTF (Forms)
  - Flask-Login, Flask-Principal, Flask-Security-Too (Authentication & RBAC)
  - Werkzeug, bcrypt, passlib (Security & password hashing)
- Database & ORM
  - SQLAlchemy (Database management), pytz (Timezone handling)
- Templating & Utilities
  - Jinja2 (Templating engine), WTForms (Form validation), email_validator (Email validation)
- Task Queue & Caching
  - Celery (Asynchronous task queue), Redis (Caching & message broker)
- Mail Service
  - Mailhog (Email testing & debugging)
- Miscellaneous
  - aniso8601 (Date/time parsing), dnspython (DNS handling), itsdangerous (Data security)

Database Schema Design (ER diagram)



API Resource Endpoints

User Authentication

- **Home:** / – Redirects to login page.
- **Customer Signup:** /register/customer – Register a new customer.
- **Professional Signup:** /register/professional – Register a new service professional.
- **Login:** /login – User login (customer, professional, admin).
- **Logout:** /logout – Logs out the user and clears session data.

Admin Endpoints

- **Dashboard:** /admin/dashboard – Overview of categories & products.
- **Search Services:** /admin/search – Search functionality for services.
- **Add Category:** /admin/category/add – Create a new category.
- **Edit Category:** /admin/category/edit/:id – Update category details.
- **Delete Category:** /admin/category/delete/:id – Delete a category.
- **Add Product:** /admin/product/add – Add a new product.
- **Edit Product:** /admin/services/update/:id – Update product details.
- **Delete Product:** /admin/product/delete/:id – Remove a product.
- **Summary:** /admin/summary – View admin service statistics.
- **Download Report:** /admin/downloadReport – Download service reports.

Professional Endpoints

- **Dashboard:** /professional/dashboard – View pending, completed, and rejected service requests.
- **Search Requests:** /professional/search – Search for service requests.
- **Summary:** /professional/summary – View service ratings & statistics.
- **Accept Request:** /professional/accept-service/:id – Accept a pending request.
- **Complete Request:** /professional/complete-service/:id – Mark request as completed.
- **Reject Request:** /professional/reject-service/:id – Reject a request.

Customer Endpoints

- **Dashboard:** /customer/dashboard – View service requests & available services.

- **Search Services:** /customer/search – Search for services & professionals.
- **Request Service:** /customer/request-service – Submit a new service request.
- **View Requested Services:** /customer/requested-services – List all service requests.
- **Submit Rating View:** /customer/service_requests – View services available for rating.
- **Submit Rating:** /customer/submit_rating – Rate a completed service.
- **Get Professionals by Category:** /get_professionals/:category – List professionals by category.
- **Get Services by Category:** /get_services/:category – List services by category.

Presentation Video

- A drive link to the presentation video, which demonstrates the features and functionality of the grocery store application, can be accessed here.
  https://drive.google.com/file/d/1rmHyqvv8Y8Ut4HD6yMq6rZKl7Dgvz1EB/view?usp=sharing